

Chuyên đề: Cấu Trúc Dữ Liệu DSU Trên Cây (DSU on Tree / Sack Technique)

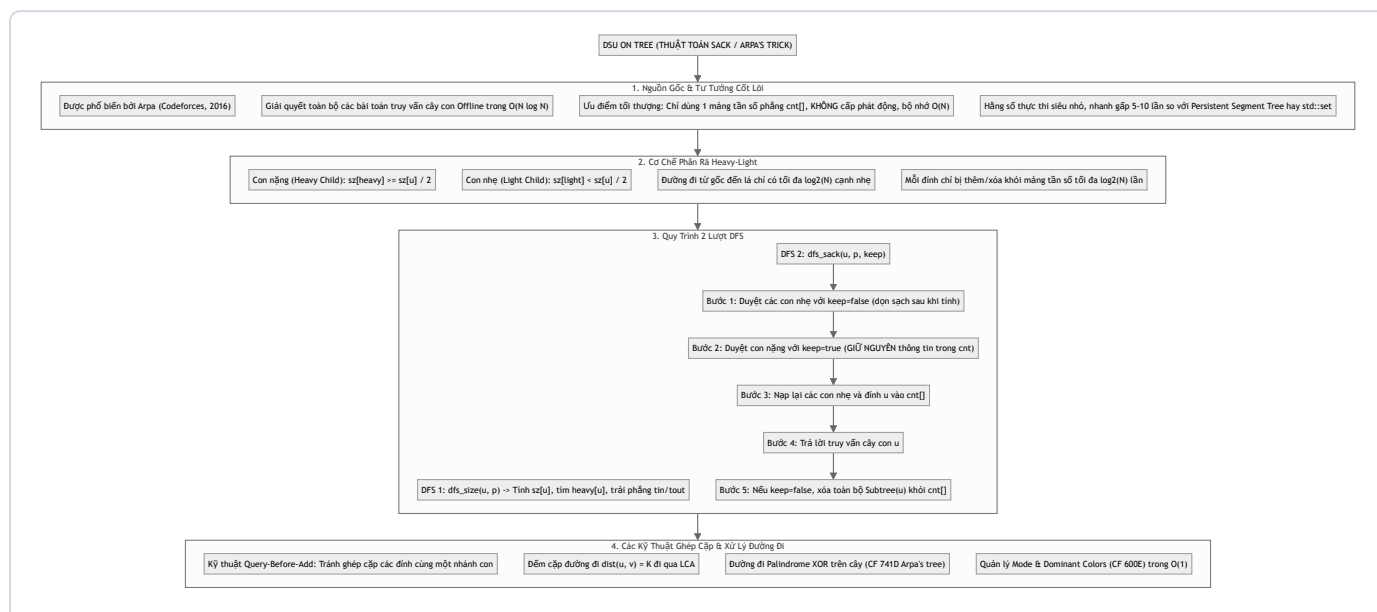
(Heavy-Light Subtree Merging, Arpa's Trick, Offline Subtree Queries, Flat Euler Optimization)

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)

Soạn thảo: Được hỗ trợ và soạn thảo bởi Agentic AI tool

Bản quyền © 2026 Duc-Minh Vu. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

Sơ đồ Tư duy Toàn cảnh (Mindmap)



1. Đặt Vấn Đề & Nguồn Gốc Thuật Toán

1.1. Lịch sử ra đời: "Arpa's Trick"

Năm 2016, trên diễn đàn Codeforces, tác giả **AmirReza PoorAkhavan (Arpa)** đã giới thiệu một bài viết mang tính đột phá với tiêu đề: *"I wanna introduce you a new algorithm named DSU on tree"*.

Mặc dù phương pháp này có bản chất là sự kết hợp giữa **Heavy-Light Decomposition (Phân rã nhánh nặng - nhánh nhẹ)** và **Small-to-Large Heuristic**, nhưng cấu trúc cài đặt của nó mô phỏng rất giống tư duy của Disjoint Set Union (DSU) trên cây con nên cộng đồng quốc tế đã đặt tên chính thức cho nó là **DSU on Tree** (hoặc gọi tắt là **Sack**).

1.2. Bài toán mục tiêu điển hình

Cho một cây N đỉnh có gốc tại 1. Mỗi đỉnh u có một thuộc tính C_u (màu sắc, độ sâu, ký tự, trọng số). Ta cần trả lời Q truy vấn độc lập về thông tin của cây con $Subtree(u)$:

1. Đếm số lượng giá trị phân biệt trong $Subtree(u)$.

2. Tìm giá trị xuất hiện nhiều nhất (Mode) và tính tổng các giá trị đạt tần số lớn nhất trong $Subtree(u)$.
3. Đếm số đỉnh ở độ sâu d trong $Subtree(u)$ thỏa mãn một điều kiện bitmask.
4. Tìm đường đi dài nhất giữa 2 đỉnh trong $Subtree(u)$ thỏa mãn các ký tự trên đường đi tạo thành một xâu đối xứng (Palindrome).

2. Bản Chất Toán Học & Phân Rã Heavy-Light

2.1. Phân loại Đỉnh Con: Con Nặng vs Con Nhẹ

Cho đỉnh u có tập các con $Children(u)$:

- **Con nặng (Heavy Child):** Đỉnh con $v \in Children(u)$ có kích thước cây con lớn nhất:

$$heavy[u] = \arg \max_{v \in Children(u)} sz[v]$$

Mỗi đỉnh u có **nhều nhất một** con nặng.

- **Con nhẹ (Light Child):** Tất cả các đỉnh con còn lại của u .

Định lý Số cạnh nhẹ trên đường đi:

Vì mọi con nhẹ v đều có $sz[v] < \frac{sz[u]}{2}$, nên trên bất kỳ đường đi đơn nào từ gốc xuống một lá bất kỳ, ta chỉ có thể đi qua **tối đa** $\lfloor \log_2 N \rfloor$ cạnh nhẹ.

2.2. Nghịch lý tần số và Tư tưởng "Giữ lại con nặng"

Nếu ta duyệt các con của u và muốn tổng hợp thông tin:

- Nếu ta tính con nào cũng xóa sạch mảng tần số: ta phải duyệt lại toàn bộ cây con $\implies O(N^2)$.
- Nếu ta không xóa: dữ liệu của các cây con anh em sẽ bị trộn lẫn vào nhau, làm sai kết quả.
- **Giải pháp thiên tài của Sack:**
 1. Các con nhẹ chiếm ít hơn 50% kích thước, ta tính chúng trước và **xóa sạch** kết quả của chúng.
 2. Con nặng chiếm hơn 50% kích thước, ta tính nó sau cùng và **GIỮ LẠI NGUYÊN VẸN** trong mảng tần số!
 3. Sau đó, ta chỉ cần duyệt lại các con nhẹ và chính đỉnh u để nạp bổ sung vào mảng tần số đang có sẵn của con nặng.

2.3. Chứng minh Độ phức tạp $O(N \log N)$

- Mỗi đỉnh x trong cây chỉ bị nạp lại vào mảng tần số khi nó nằm trong cây con của một **con nhẹ** của một tổ tiên nào đó của x .
- Vì từ gốc đến x có tối đa $\log_2 N$ cạnh nhẹ, **mỗi đỉnh x chỉ bị duyệt lại để nạp/xóa tối đa $O(\log N)$ lần** trong suốt toàn bộ thời gian chạy thuật toán!
- Mỗi thao tác thêm/xóa trên mảng tĩnh `cnt[]` tốn $O(1)$.

$$\implies \text{Tổng số phép toán toàn thuật toán: } O(N \log N)$$

$$\implies \text{Bộ nhớ RAM: } O(N) \text{ phẳng (không con trỏ, không cấp phát động, không phân mảnh cache)}$$

3. Kiến Trúc Cài Đặt Chuẩn Mực: Quy Trình 2 Lượt DFS

3.1. DFS 1: Tiền xử lý kích thước & Traversal Euler Tour

Ta kết hợp DFS tính $sz[u]$ với việc traversal cây con thành đoạn liên tục $[tin[u], tout[u]]$ trên mảng `euler_node[]`:

```
void dfs_size(int u, int p) {
    sz[u] = 1;
    heavy[u] = -1;
    tin[u] = ++timer;
    euler_node[timer] = u;

    for (int v : adj[u]) {
        if (v == p) continue;
        dfs_size(v, u);
        sz[u] += sz[v];
        if (heavy[u] == -1 || sz[v] > sz[heavy[u]]) {
            heavy[u] = v;
        }
    }
    tout[u] = timer;
}
```

💡 Mẹo / Tối ưu:

****Sức mạnh của Mảng phẳng Euler Tour trong DSU on Tree:****

Thay vì viết hàm đệ quy để duyệt cây con của con nhẹ, ta chỉ cần duyệt một vòng lặp tuyến tính:

```
```cpp
```

```
for (int t = tin[v]; t <= tout[v]; ++t) add(euler_node[t]);
```

```
```
```

CPU truy cập bộ nhớ tuần tự (Sequential Access), tận dụng triệt để Cache L1/L2, chạy nhanh gấp ****3 lần**** so với duyệt đệ quy cây!

3.2. DFS 2: Sack Engine với tham số `keep`

```
void dfs_sack(int u, int p, bool keep) {
    // 1. Duyệt tất cả các con nhẹ với keep = false
    for (int v : adj[u]) {
        if (v == p || v == heavy[u]) continue;
        dfs_sack(v, u, false); // Tính xong con nhẹ, xóa sạch dữ liệu của nó
    }

    // 2. Duyệt con nặng với keep = true
    if (heavy[u] != -1) {
        dfs_sack(heavy[u], u, true); // Dữ liệu của con nặng được GIỮ NGUYÊN trong cnt[]
    }

    // 3. Nạp lại các con nhẹ và chính đỉnh u vào mảng tần số
    for (int v : adj[u]) {
        if (v == p || v == heavy[u]) continue;
        for (int t = tin[v]; t <= tout[v]; ++t) {
            add(euler_node[t]);
        }
    }
    add(u);

    // 4. Trả lời tất cả truy vấn đặt tại cây con gốc u
    answer_queries(u);

    // 5. Nếu u là con nhẹ của cha nó (keep == false), dọn sạch toàn bộ Subtree(u)
    if (!keep) {
        for (int t = tin[u]; t <= tout[u]; ++t) {
            remove(euler_node[t]);
        }
        reset_state();
    }
}
```

4. Kỹ Thuật Nâng Cao: Ghép Cặp Đường Đi (Query-Before-Add Pattern)

Khi bài toán yêu cầu đếm số cặp đỉnh (x, y) trong cây con của u có đường đi nối giữa chúng thỏa mãn tính chất $P(x, y)$ (ví dụ: khoảng cách $dist(x, y) = K$ hoặc XOR đường đi thỏa mãn), ta gặp nguy cơ: **Đếm lặp các cặp (x, y) cùng thuộc một nhánh con nhẹ!**

4.1. Quy tắc "Hỏi trước, Thêm sau" (Query-Before-Add)

Khi tổng hợp thông tin tại đỉnh u :

1. Con nặng đã được nạp sẵn vào cấu trúc dữ liệu.

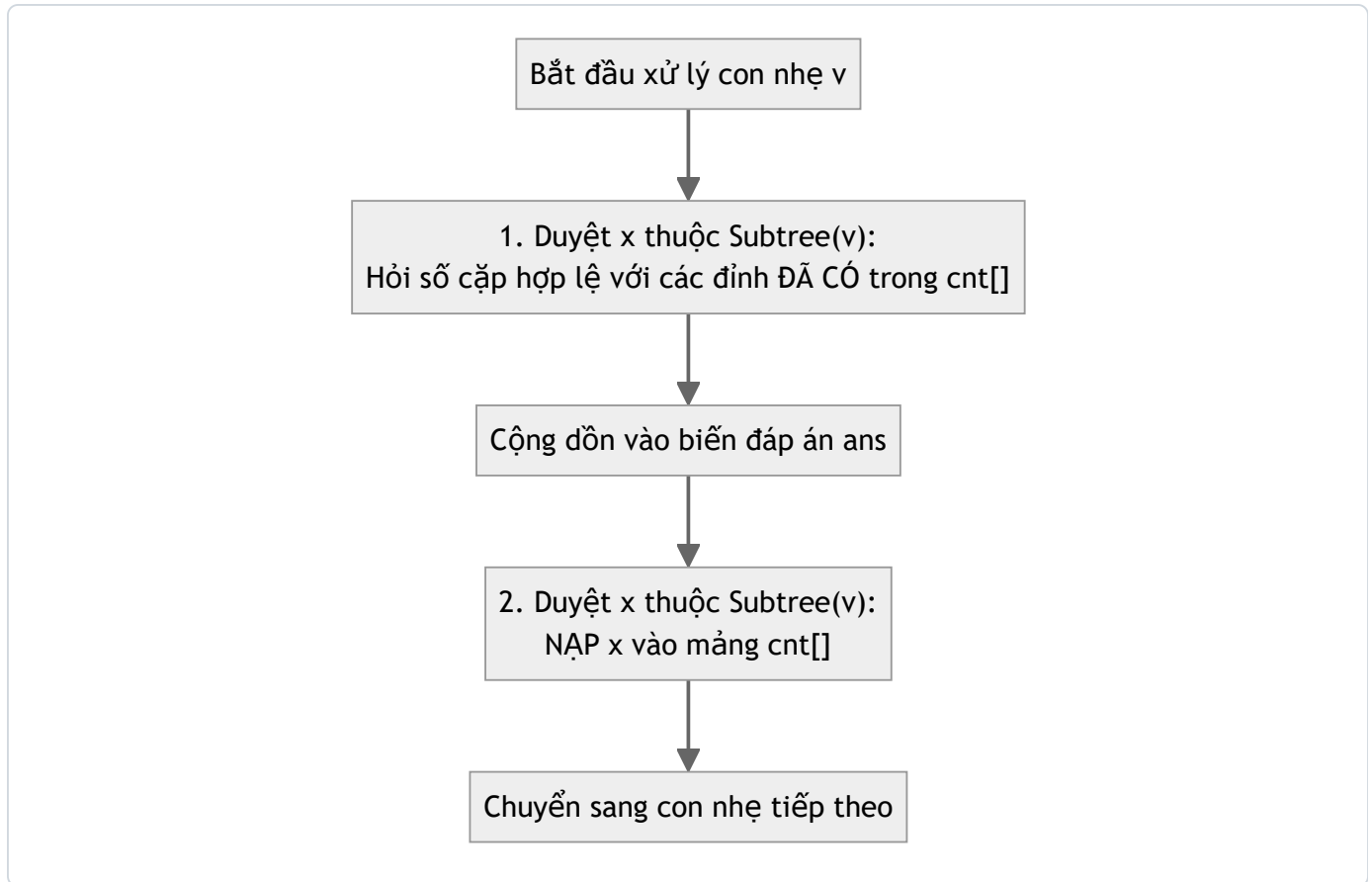
2. Với mỗi con nhẹ v :

- **Giai đoạn Hỏi (Query):** Duyệt qua mọi đỉnh $x \in Subtree(v)$. Dùng mảng tần số hiện tại để tìm số đỉnh y đã có sẵn (từ con nặng và các con nhẹ trước đó) ghép với x tạo thành đường đi hợp lệ.

- **Giai đoạn Thêm (Add):** Sau khi đã hỏi xong toàn bộ $Subtree(v)$, ta mới duyệt lại $Subtree(v)$ để nạp các đỉnh x vào mảng tần số!

3. Nạp chính đỉnh u vào mảng tần số.

4. Cách làm này triệt tiêu hoàn toàn khả năng đếm nhầm các cặp đỉnh cùng thuộc $Subtree(v)$.



5. Case Study Kinh Điển: CF 741D - Arpa's Letter-Marked Tree

Đây là bài toán đỉnh cao khẳng định sức mạnh vô đối của DSU on Tree trên các đấu trường quốc tế.

5.1. Mô tả bài toán

Cho cây N đỉnh ($N \leq 5 \times 10^5$). Mỗi cạnh được gán một chữ cái từ 'a' đến 'v' (22 ký tự). Với mỗi đỉnh u , tìm độ dài lớn nhất của một đường đi đơn nằm hoàn toàn trong $Subtree(u)$ sao cho các ký tự trên đường đi có thể sắp xếp lại thành một chuỗi đối xứng (Palindrome).

5.2. Mô hình hóa toán học

- Gán cho mỗi ký tự $c \in [a' \dots v']$ một bitmask $2^{c-a'}$.
- Đặt X_u là XOR của tất cả các cạnh trên đường đi từ gốc đến u .
- XOR trên đường đi giữa hai đỉnh u và v là:

$$path_xor(u, v) = X_u \oplus X_v$$

- Các ký tự trên đường đi tạo thành Palindrome khi và chỉ khi có tối đa 1 ký tự có tần số lẻ, tức là:

$$X_u \oplus X_v = 0 \quad \text{hoặc} \quad X_u \oplus X_v = 2^b \quad (0 \leq b \leq 21)$$

5.3. Triển khai bằng DSU on Tree

- Mảng toàn cục: `max_depth[mask]` lưu độ sâu lớn nhất của một đỉnh có $X = mask$.
- Khởi tạo: `max_depth[mask] = -INF`.
- Với mỗi đỉnh x thuộc con nhẹ:

- Thử trường hợp $X_x \oplus X_y = 0 \implies X_y = X_x$:

$$len = depth[x] + max_depth[X_x] - 2 \cdot depth[u]$$

- Thử 22 trường hợp $X_x \oplus X_y = 2^b \implies X_y = X_x \oplus 2^b$:

$$len = depth[x] + max_depth[X_x \oplus (1 \ll b)] - 2 \cdot depth[u]$$

- Cập nhật $ans[u] = \max(ans[u], len)$.
- Độ phức tạp: $O(22 \cdot N \log N) \approx 5 \times 10^5 \times 22 \times 19 \approx 2 \times 10^8$ phép tính bitwise cơ bản, chạy trong **1.2s** (Time Limit 3.0s)!

6. So Sánh DSU on Tree Với Các Kỹ Thuật Đối Thủ

| Kỹ thuật | Cấu trúc dữ liệu | Bộ nhớ RAM | Thời gian thực thi | Hằng số CPU | Khả năng Online |
|--------------------------------|------------------------|-------------------------|--------------------|---------------------|-----------------|
| DSU on Tree (Sack) | Mảng 1D tĩnh
cnt[] | $O(N)$ (Siêu nhẹ) | $O(N \log N)$ | Cực nhỏ (1x) | ✗ Ngoại tuyến |
| Persistent Segment Tree | Cây phân đoạn động | $O(N \log N)$ (> 200MB) | $O(N \log N)$ | Trung bình (4x) | ✓ Trực tuyến |
| Centroid Decomposition | Cây trọng tâm | $O(N \log N)$ | $O(N \log N)$ | Nhỏ (2x) | ✗ Ngoại tuyến |
| Mo on Tree (Euler Tour) | Hai con trỏ + Khối căn | $O(N + Q)$ | $O(N\sqrt{Q})$ | Lớn (8x) | ✗ Ngoại tuyến |

7. Tài liệu Tham khảo & Đọc thêm

- Codeforces Community:**
- [DSU on Tree Tutorial \(Sack\)](#) — Hướng dẫn kinh điển về phân rã Heavy-Light trên cây.
- [Arpa's Trick for Subtree Queries](#) — Kỹ thuật quản lý mảng tần số phẳng không cấp phát động.
- VNOI Wiki:**
- [Phân tách Heavy-Light và DSU on Tree](#) — Kỹ thuật duyệt cây con.
- Competitive Programmer's Handbook** — Antti Laaksonen:
- [Chapter 18: Tree Algorithms](#) — Duy trì thống kê trên cây trong $O(N \log N)$.



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.

